example. Also, the makeup of your team and the overall efficiency might change if you had to build more than one house. Even though your plumber might not have lots of work to do on any given house, you might have enough work to keep him busy all the time if you're building many houses at once. Also, if you don't have to pay your specialists when there's no work for them to do, you might be able to afford a larger team overall even if you have only the same number of people working at any one time.

OK, enough about building; what does all this have to do with threads? Well, with threads the same issues apply. You need to decide how many threads to use and what tasks they should be doing. You need to decide whether to have "generalist" threads that do whatever work is necessary at any point in time or "specialist" threads that do one thing well, or some combination. You need to make these choices whatever the driving reason for using concurrency, and quite how you do this will have a crucial effect on the performance and clarity of the code. It's therefore vital to understand the options so you can make an appropriately informed decision when designing the structure of your application. In this section, we'll look at several techniques for dividing the tasks, starting with dividing data between threads before we do any other work.

8.1.1. Dividing data between threads before processing begins

The easiest algorithms to parallelize are simple algorithms such as `std::for_each` that perform an operation on each element in a data set. In order to parallelize such an algorithm, you can assign each element to one of the processing threads. How the elements are best divided for optimal performance depends very much on the details of the data structure, as you'll see later in this chapter when we look at performance issues.

The simplest means of dividing the data is to allocate the first *N* elements to one thread, the next *N* elements to another thread, and so on, as shown in figure 8.1, but other patterns could be used too. No matter how the data is divided, each thread then processes just the elements it has been assigned without any communication with the other threads until it has completed its processing.